

Principles of Distributed Systems

By: Fahad Sabah

计算机学院,

北京工业大学, 中国

**College of Computer Science,
Beijing University of Technology, China**

fahad.sabah@bjut.edu.cn



Contents

1. What is a distributed system?

2. Why do we build it?

3. What are the Core Principles?

4. What major types exist?

5. Project Discussion

目录

1. 什么是分布式系统？

2. 我们为什么要建造它？

3. 核心原则是什么？

4. 有哪些主要类型存在？

5. 项目讨论

Distributed System: Definition [分布式系统：定义]

A distributed system is:

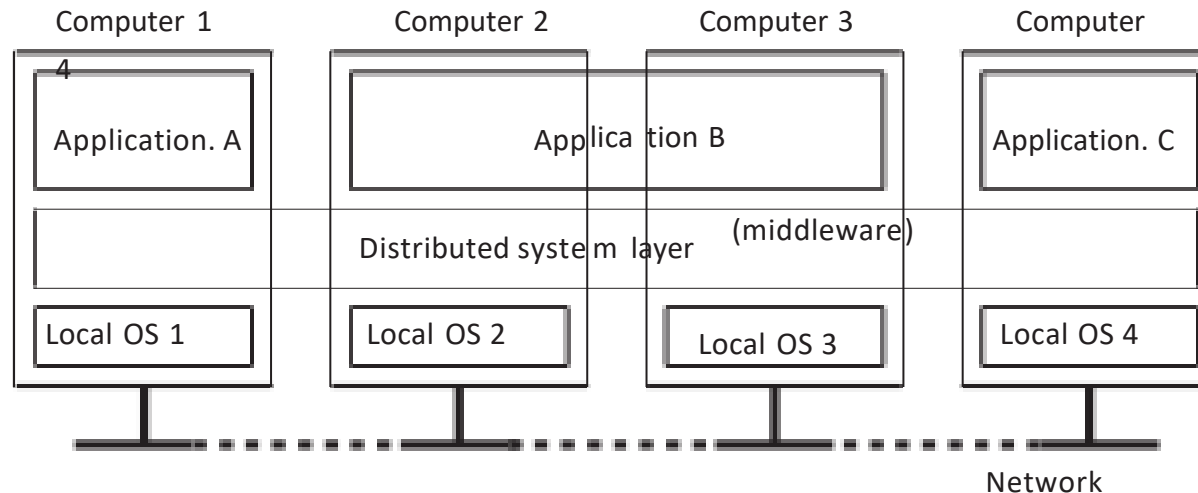
*a collection of **independent computers**
appears to its users as a **single coherent system***

Two aspects: (1) independent computers and (2) single system → **middleware**.

分布式系统包括：

一组独立计算机在用户眼中呈现为一个统一的系统

两个方面：（1）独立计算机，（2）单系统→**中间件**。



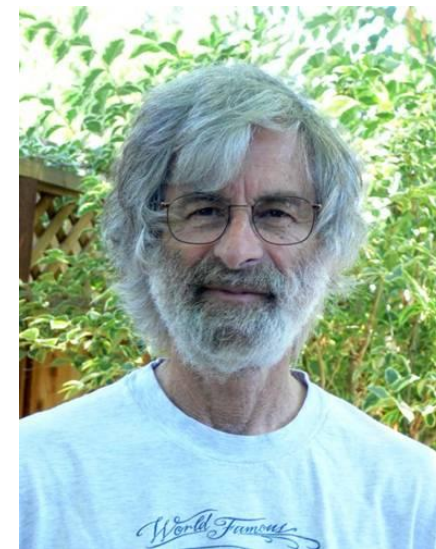
Distributed System: Definition [分布式系统：定义]

"A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable."

“分布式系统是指你甚至不知道存在的计算机故障，可能导致你自己的电脑无法使用。”



-- Leslie Lamport

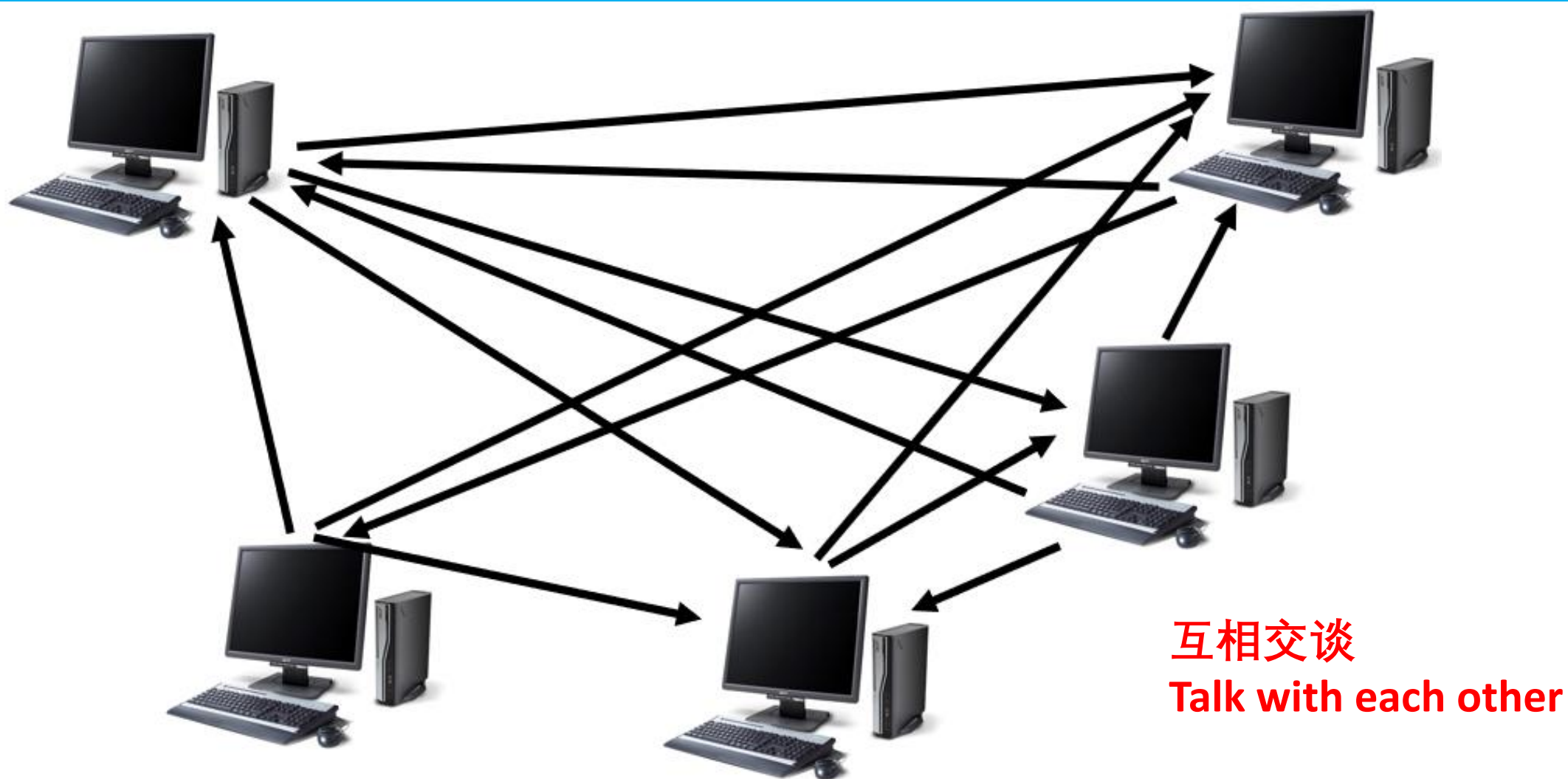


What is a Distributed System? [什么是分布式系统?]

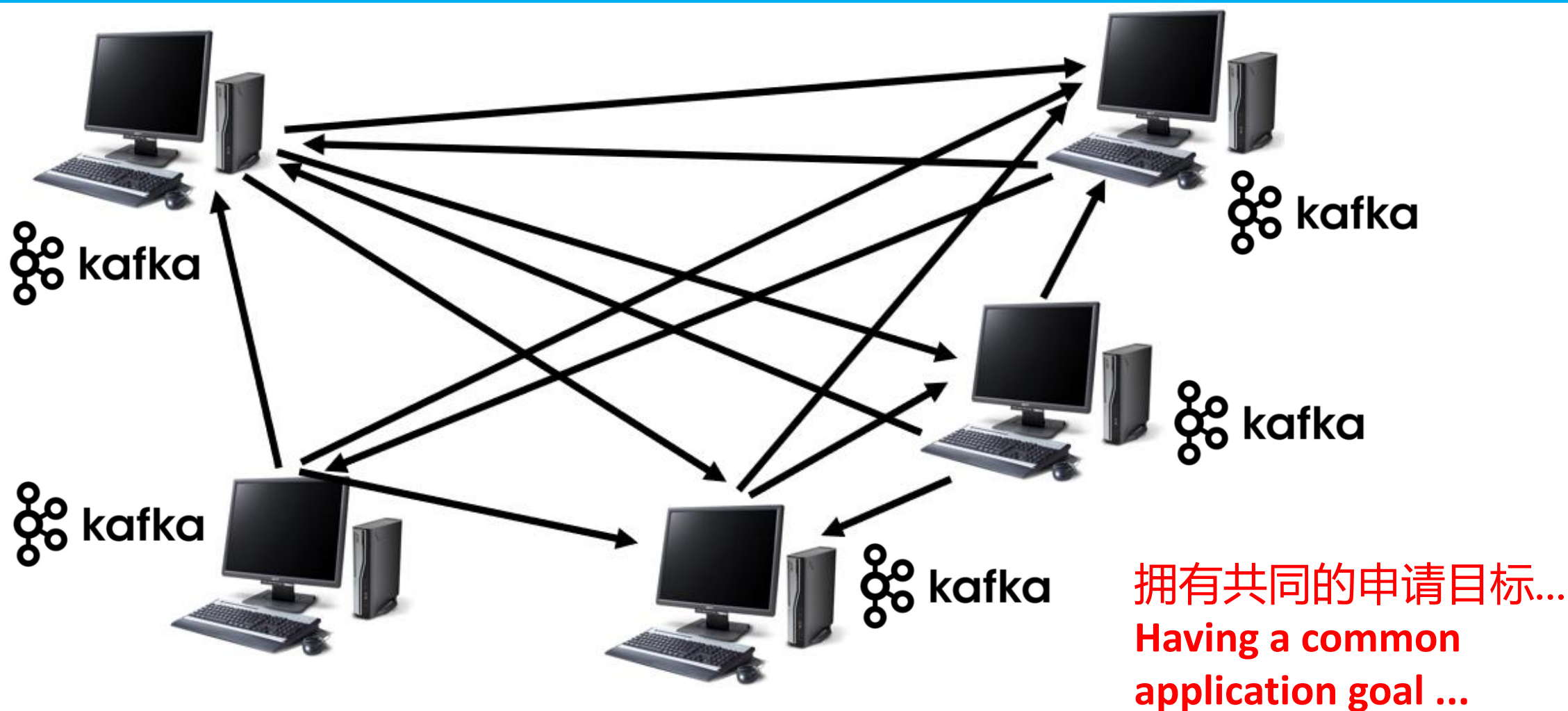


多台计算机
Multiple Computers

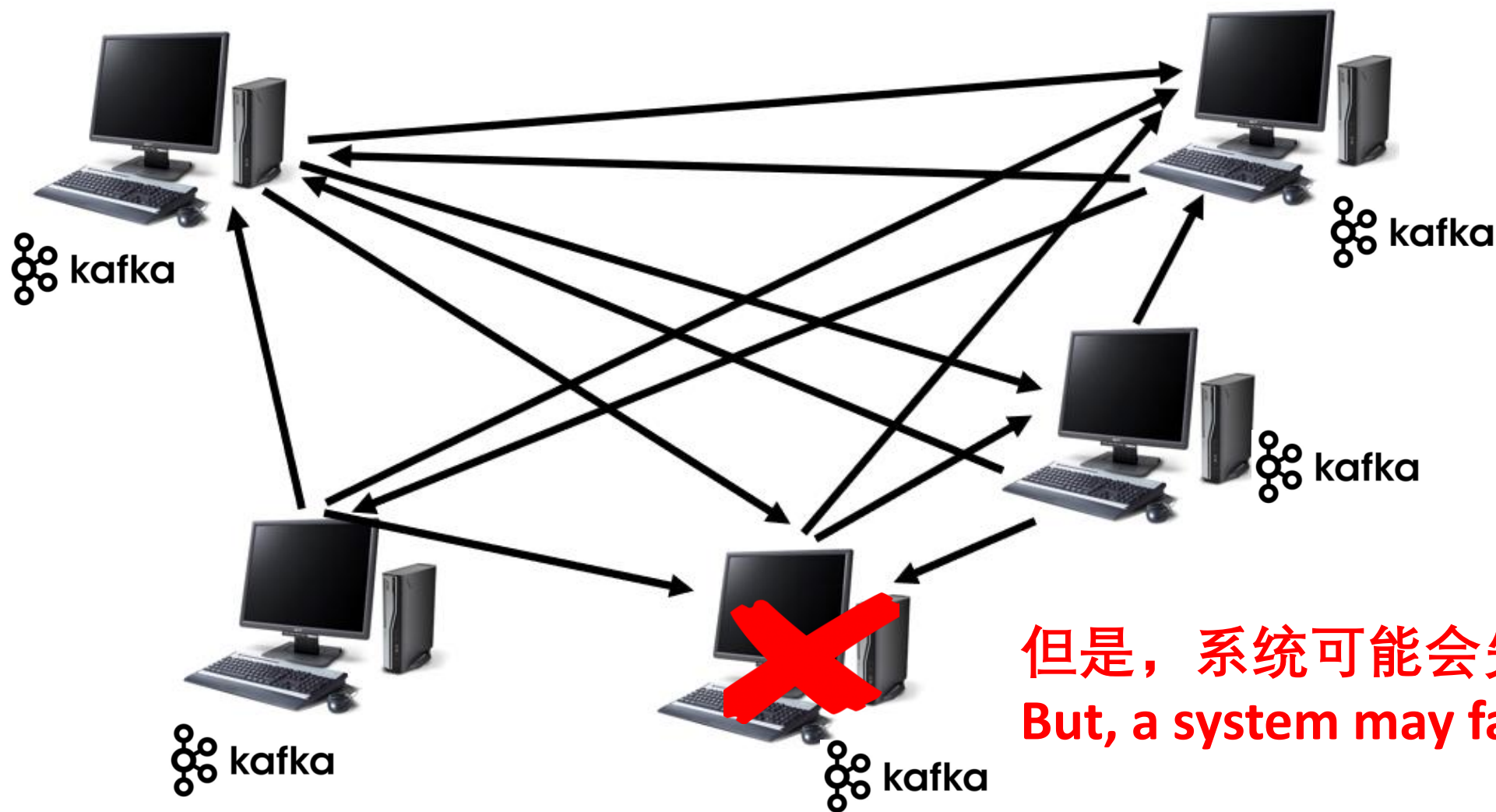
What is a Distributed System? [什么是分布式系统?]



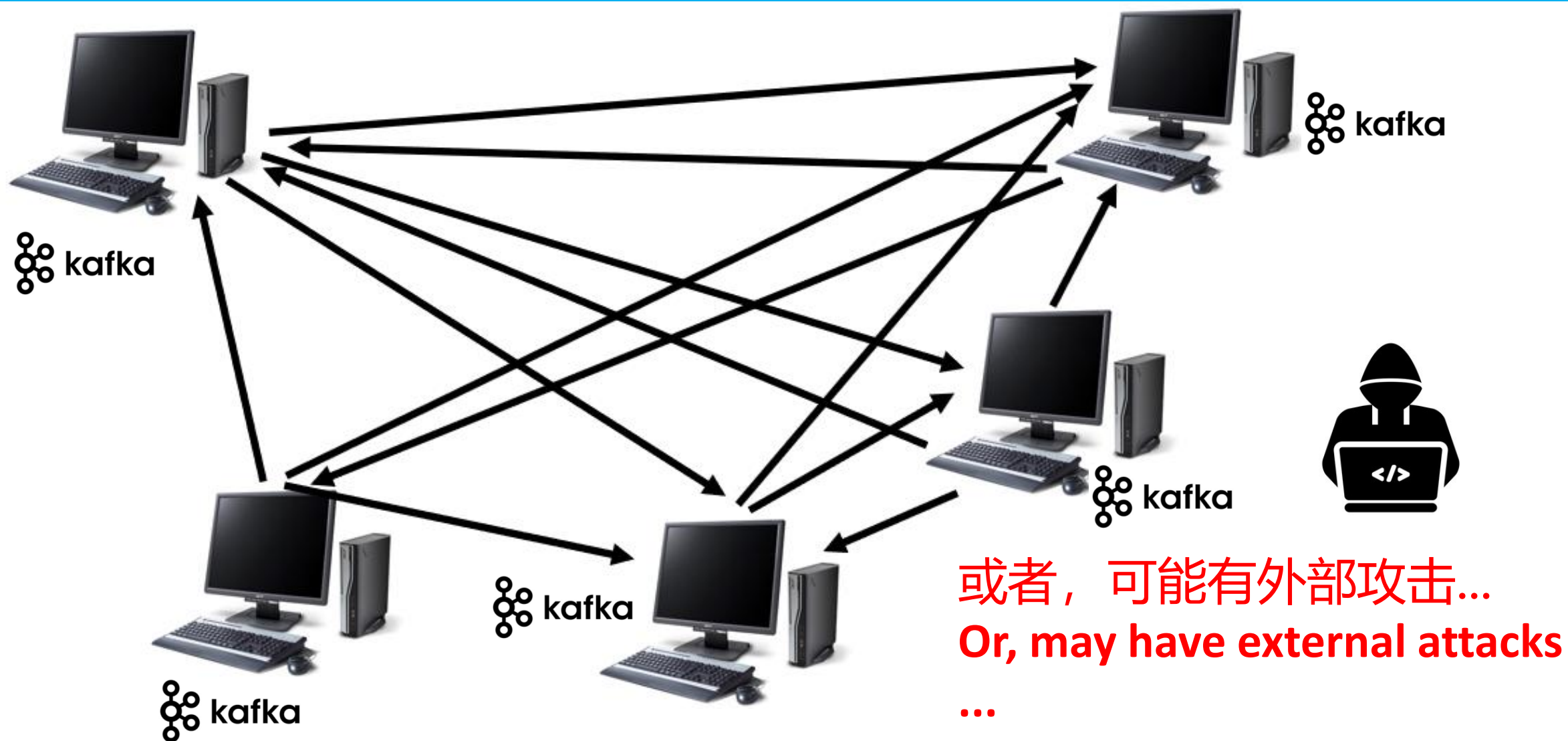
What is a Distributed System? [什么是分布式系统?]



What is a Distributed System? [什么是分布式系统?]



What is a Distributed System? [什么是分布式系统?]



Examples of Distributed System[分布式系统的示例]

- Almost every large system that you use ...

几乎所有大型系统.....



淘宝
Taobao



Distributed Systems Evolution

(Early 1960s) The Conceptual Seed

- ❑ Paul Baran's paper in 1962 formally introducing the term
- ❑ Licklider's "Galactic Network" vision in 1962
- ❑ Leonard Kleinrock's paper on distributed network theory in 1965

(Late 1960s) First Real-World Implementation

- ❑ the launch of ARPANET in 1969, the first operational packet-switching network and the foundation of today's internet

(Early 1970s) Early Practical Systems

- ❑ first distributed file system was documented in 1972
- ❑ The first Ethernet networks in 1973 showed local computer clusters could share data

A Foundational Paper (1978):

- ❑ The paper "Time, Clocks, and the Ordering of Events in a Distributed System" by Leslie Lamport is often called the "birth certificate" of distributed computing as a formal academic field

How does Distributed Systems Work?

Three Core Principles

Task Distribution: Split one big job into smaller jobs for many computers.

❑ 任务分配： 把一个大任务拆成多个小任务， 交给多台计算机。

❑ Data Partitioning: Split large data into smaller parts and store them on different machines.

❑ 数据分片： 把大数据拆成小块， 存到不同机器上。

❑ Synchronization and Coordination: Make sure all computers work together in the correct order.

❑ 同步与协调： 保证所有计算机按正确顺序协同工作。

分布式系统是如何工作的？

How does Distributed Systems Work?

Core Principles

Think of it like a big school group project:

1. Task Distribution → ☐ one student does not do all the work, the work is divided
2. Data Partitioning → ☐ the information is divided
3. Synchronization and Coordination → ☐ everyone must still stay organized

分布式系统是如何工作的？

How does Distributed Systems Work?

Core Principles

1. Task Distribution
2. Data Partitioning
3. Synchronization and Coordination

Baidu search example

- ☐ one machine receives your search
- ☐ some machines search web pages
- ☐ some rank results
- ☐ some return the final answer

So the work is distributed.

分布式系统是如何工作的？

How does Distributed Systems Work?

Core Principles

1. Task Distribution
2. Data Partitioning
3. Synchronization and Coordination

Easy analogy Like a library:

one room keeps books A–F

another room keeps books G–M

another room keeps books N–Z

WeChat / Alibaba user data

- ☐ user data is too large for one database
- ☐ so users are divided into groups
- ☐ one server stores one group
- ☐ another server stores another group

This is often called sharding.

分布式系统是如何工作的？

How does Distributed Systems Work?

Core Principles

1. Task Distribution
2. Data Partitioning
3. Synchronization and Coordination

Easy analogy Like traffic lights:

if everyone moves without rules, there is chaos

if signals are coordinated, traffic flows safely

Online shopping on Taobao

- ☐ one service checks stock
- ☐ one service handles payment
- ☐ one service creates the order
- ☐ one service updates delivery

If they do not coordinate:

- ☐ payment may succeed but order may fail
- ☐ stock may become wrong
- ☐ two users may buy the last item at the same time

So the system needs synchronization and coordination.

Pre-requisite for this course [本课程的先修课程]

Data structures & algorithms

数据结构与算法

Trees, hashing,
partitioning, time
complexity

Operating systems

操作系统

Processes, threads,
memory, scheduling

Computer networks

计算机网络

Protocols, latency,
routing, congestion

Database systems

数据库系统

Transactions,
consistency,
replication, queries

Course Contents/Tentative Plan [课程内容/暂定计划]

Week

01: Introduction

02: Architectures

03: Processes

04: Communication

05: Naming

06: Synchronization

07: Consistency & Replication

08: Fault Tolerance

09: Security

10: Distributed Object-Based Systems

11: Distributed File and Web-Based Systems

12: Distributed Coordination Systems

一周

01: 介绍

02: 建筑

03: 流程

04: 沟通

05: 命名

06: 同步

07: 一致性与复制

08: 容错

09: 安全

10: 基于对象的分布式系统

11: 分布式文件与基于网页的系统

12: 分布式协调系统

Course Contents/Tentative Plan [课程内容/暂定计划]

Week

01: Introduction

02: Architectures

03: Processes

04: Communication

05: Naming

06: Synchronization

07: Consistency & Replication

08: Fault Tolerance

09: Security

10: Distributed Object-Based Systems

11: Distributed File and Web-Based Systems

12: Distributed Coordination Systems

DEFINITION OF A DISTRIBUTED SYSTEM

1.2 GOALS

1.2.1 Making Resources Accessible

1.2.2 Distribution Transparency

1.2.3 Openness

1.2.4 Scalability

1.2.5 Pitfalls

1.3 TYPES OF DISTRIBUTED SYSTEMS

1.3.1 Distributed Computing Systems

1.3.2 Distributed Information Systems

1.3.3 Distributed Pervasive Systems

Week

01: Introduction

02: Architectures

03: Processes

04: Communication

05: Naming

06: Synchronization

07: Consistency & Replication

08: Fault Tolerance

09: Security

10: Distributed Object-Based Systems

11: Distributed File and Web-Based Systems

12: Distributed Coordination Systems

2.1 ARCHITECTURAL STYLES

2.2 SYSTEM ARCHITECTURES

2.2.1 Centralized Architectures

2.2.2 Decentralized Architectures

2.2.3 Hybrid Architectures

2.3 ARCHITECTURES VERSUS MIDDLEWARE

2.3.1 Interceptors

2.3.2 General Approaches to Adaptive Software

2.3.3 Discussion

2.4 SELF-MANAGEMENT IN DISTRIBUTED SYSTEMS

2.4.1 The Feedback Control Model

2.4.2 Example: Systems Monitoring with Astrolabe

2.4.3 Example: Differentiating Replication Strategies in Globule

2.4.4 Example: Automatic Component Repair Management in Jade

Course Contents/Tentative Plan [课程内容/暂定计划]

Week

01: Introduction

02: Architectures

03: Processes

04: Communication

05: Naming

06: Synchronization

07: Consistency & Replication

08: Fault Tolerance

09: Security

10: Distributed Object-Based Systems

11: Distributed File and Web-Based
Systems

12: Distributed Coordination Systems

3.1 THREADS

3.1.1 Introduction to Threads

3.1.2 Threads in Distributed Systems

3.2 VIRTUALIZATION

3.2.1 The Role of Virtualization in Distributed Systems

3.2.2 Architectures of Virtual Machines

3.3 CLIENTS

3.3.1 Networked User Interfaces

3.3.2 Client-Side Software for Distribution Transparency

3.4 SERVERS

3.4.1 General Design Issues

3.4.2 Server Clusters

3.4.3 Managing Server Clusters

3.5 CODE MIGRATION

3.5.1 Approaches to Code Migration

3.5.2 Migration and Local Resources

3.5.3 Migration in Heterogeneous Systems

Course Contents/Tentative Plan [课程内容/暂定计划]

Week

01: Introduction

02: Architectures

03: Processes

04: Communication

05: Naming

06: Synchronization

07: Consistency & Replication

08: Fault Tolerance

09: Security

10: Distributed Object-Based Systems

11: Distributed File and Web-Based Systems

12: Distributed Coordination Systems

4.1 FUNDAMENTALS

4.1.1 Layered Protocols

4.1.2 Types of Communication

4.2 REMOTE PROCEDURE CALL

4.2.1 Basic RPC Operation

4.2.2 Parameter Passing

4.2.3 Asynchronous RPC

4.2.4 Example: DCE RPC

4.3 MESSAGE-ORIENTED COMMUNICATION

4.3.1 Message-Oriented Transient Communication

4.3.2 Message-Oriented Persistent Communication

4.3.3 Example: ffiM's WebSphere Message-Queuing System

4.4 STREAM-ORIENTED COMMUNICATION

4.4.1 Support for Continuous Media

4.4.2 Streams and Quality of Service

4.4.3 Stream Synchronization

4.5 MULTICAST COMMUNICATION

4.5.1 Application-Level Multicasting

4.5.2 Gossip-Based Data Dissemination

Course Grading [课程评分]

□ Assignments/Quizzes: 30%

□ Project(s): 70%

□ Mid: 30%

□ Final: 40%

□ 作业/测验: 30%

□ 项目: 70%

□ 中游: 30%

□ 最终: 40%

Course Structure [课程结构]

- ❑ A blend of lecture-based study and research-based study with Programming Assignments / Research Projects
- ❑ Lectures
 - ❑ Discuss the fundamentals – algorithms and systems-aspect of the course
- ❑ Term Projects
 - ❑ Two types of term projects – Survey-based and Research-based
 - ❑ Survey-based: Read some (at least 20x papers for x-member groups) recent research papers on a topic assigned and come-up with a survey paper
 - ❑ Research-based: Explore a recent research topic and come up with some experimental studies/new algorithms/new system designs, etc.
 - ❑ Group size: 2 to 3 members
 - ❑ QR for project registration will be shared later

Term Projects [学期项目]

Two Types of Term Projects: Survey-based and Research-based
两种类型的期末项目：调研型 和 研究/技术型



MIDTERM PROJECT Make Survey 调研型项目 (期中)

Survey-based: Read recent research papers on an assigned topic and produce a survey paper.
调研型：阅读近期研究论文，并撰写调研论文。

How it Works / 如何进行



1. Read Papers
阅读论文
(At least 20×)
至少20篇
2. Analyze & Organize
分析与整理
Themes, methods, results, gaps
主题、方法、结果、空白
3. Write Survey Paper
撰写调研论文
Summarize and compare key works
总结并比较关键工作
4. Present
展示汇报
Slides + Q&A
幻灯片 + 问答

Example Topics (调研主题示例)

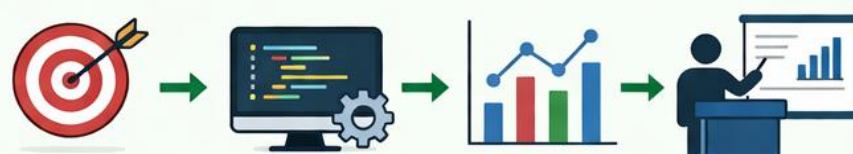
- Distributed Consensus Algorithms
分布式一致性算法
- Cloud Storage Systems
云存储系统
- Edge Computing
边缘计算
- Blockchain Scalability
区块链可扩展性



FINAL TERM PROJECT Research / Technical Project 研究/技术型项目 (期末)

Research-based: Explore a recent research topic and come up with experimental studies, new algorithms, or system designs.
研究型：探索一个近期研究课题，提出实验研究、新算法或系统设计。

How it Works / 如何进行



1. Choose Topic
选择课题
Recent & meaningful topic
近期且有意义的课题
2. Design & Implement
设计与实现
Algorithms, systems or experiments
算法、系统或实验
3. Evaluate
评估实验
Run experiments, collect results
运行实验，收集结果
4. Write & Present
撰写与展示
Paper + Slides + Demo
论文 + 幻灯片 + 演示

Example Topics (研究课题示例)

- New Consensus Algorithm
新的一致性算法
- Fault-Tolerant System Design
容错系统设计
- Distributed Machine Learning
分布式机器学习
- Performance Optimization
性能优化



GROUP SIZE
小组规模
2 to 3 members
2-3 名成员



KEY REQUIREMENTS 关键要求

- Good reading and analysis (阅读与分析能力)
- Clear writing and presentation (清晰的写作与展示)
- Teamwork and contribution (团队合作与贡献)



TIMELINE
时间安排
Midterm: Survey (调研)
Final: Research/Technical (研究/技术)

Books and References [书籍与参考文献]

- ❑ van Steen 和 Tanenbaum, 《分布式系统（任何版本）》电子书: [书籍与参考资料](#)
- ❑ 我们会关注各种论文和文章, 并在讨论GitHub仓库中不同的话题时进行引用。
- ❑ van Steen and Tanenbaum, **Distributed Systems** (any edition) e-book available: [Books and Reference Materials](#)
- ❑ We'll follow various papers and articles, will refer them during discussing different topics available at the github repository.

摘要 [第一场] Summary [Session 1]

- ❑ Distributed systems are powerful, but hidden dependencies matter.
- ❑ A small failure can travel through the system and affect many users.
- ❑ This is why fault tolerance and monitoring are important topics later.

- ❑ Too many users for a single machine
- ❑ Too much data for one local disk
- ❑ Need fast response in many cities/regions
- ❑ Need to survive machine, network, or rack failures

- ❑ In 2023, Taobao reached 583,000 orders created per second
- ❑ On New Year's Eve 2017, WeChat saw 14.2 billion red packets sent/received, peaking at 760,000 per second.
- ❑ In 2021, Douyin became the official Spring Festival Gala partner, distributing ¥2 billion (~\$280M) in red packets, with 70.3 billion total interactions

Goals of Distributed Systems [分布式系统的目标]

Availability (可获得性)

Use data, services, and devices from anywhere

Distribution transparency (分发透明度)

Hide complexity from users

Openness (开放性)

Allow different systems to work together

Scalability (可扩展性)

Keep working well as users/data grow

Easy memory trick

Resources = use things anywhere

Transparency = hide the mess

Openness = follow standards

Scalability = grow without breaking

Distribution Transparency [分发透明度]

Transp.	Description
Access	Hide differences in data representation and invocation mechanisms
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide failure and possible recovery of an object

Note

Distribution transparency is a nice a goal, but achieving it is a different story.

Distribution Transparency [分发透明度]

Transp.	Description
Access	Hide differences in data representation and invocation mechanisms
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide failure and possible recovery of an object

Note

Distribution transparency is a nice a goal, but achieving it is a different story.

Degree of Transparency [透明度程度]

Observation

Aiming at full distribution transparency may be too much:

- Users may be located in **different continents**
- **Completely hiding failures** of networks and nodes is (theoretically and practically) **impossible**
 - You cannot distinguish a slow computer from a failing one
 - You can never be sure that a server actually performed an operation before a crash
- Full transparency will **cost performance**, exposing distribution of the system
 - Keeping Web caches **exactly** up-to-date with the master
 - Immediately flushing write operations to disk for fault tolerance

Degree of Transparency [透明度程度]

Observation

Aiming at full distribution transparency may be too much:

- Users may be located in **different continents**
- **Completely hiding failures** of networks and nodes is (theoretically and practically) **impossible**
 - You cannot distinguish a slow computer from a failing one
 - You can never be sure that a server actually performed an operation before a crash
- Full transparency will **cost performance**, exposing distribution of the system
 - Keeping Web caches **exactly** up-to-date with the master
 - Immediately flushing write operations to disk for fault tolerance

Degree of Transparency [透明度程度]

Observation

Aiming at full distribution transparency may be too much:

- Users may be located in **different continents**
- **Completely hiding failures** of networks and nodes is (theoretically and practically) **impossible**
 - You cannot distinguish a slow computer from a failing one
 - You can never be sure that a server actually performed an operation before a crash
- Full transparency will **cost performance**, exposing distribution of the system
 - Keeping Web caches **exactly** up-to-date with the master
 - Immediately flushing write operations to disk for fault tolerance

Degree of Transparency [透明度程度]

Observation

Aiming at full distribution transparency may be too much:

- Users may be located in **different continents**
- **Completely hiding failures** of networks and nodes is (theoretically and practically) **impossible**
 - You cannot distinguish a slow computer from a failing one
 - You can never be sure that a server actually performed an operation before a crash
- Full transparency will **cost performance**, exposing distribution of the system
 - Keeping Web caches **exactly** up-to-date with the master
 - Immediately flushing write operations to disk for fault tolerance

Openness of Distributed Systems [分布式系统的开放性]

Open distributed system

Be able to interact with services from other open systems, irrespective of the underlying environment:

Systems should conform to well-defined **interfaces**

Systems should support **portability** of applications

Systems should easily **interoperate**

Achieving openness

At least make the distributed system independent from **heterogeneity** of the underlying environment:

Hardware

Platforms

Languages

Openness of Distributed Systems [分布式系统的开放性]

Open distributed system

Be able to interact with services from other open systems, irrespective of the underlying environment:

Systems should conform to well-defined **interfaces**

Systems should support **portability** of applications

Systems should easily **interoperate**

Achieving openness

At least make the distributed system independent from **heterogeneity** of the underlying environment:

Hardware

Platforms

Languages

Policies versus Mechanisms [政策与机制]

Implementing openness

Requires support for different **policies**:

- What level of consistency do we require for client-cached data?
- Which operations do we allow downloaded code to perform?
- Which QoS requirements do we adjust in the face of varying bandwidth?
- What level of secrecy do we require for communication?

Implementing openness

Ideally, a distributed system provides only **mechanisms**:

- Allow (dynamic) setting of caching policies
- Support different levels of trust for mobile code
- Provide adjustable QoS parameters per data stream
- Offer different encryption algorithms

Policies versus Mechanisms [政策与机制]

Implementing openness

Requires support for different **policies**:

- What level of consistency do we require for client-cached data?
- Which operations do we allow downloaded code to perform?
- Which QoS requirements do we adjust in the face of varying bandwidth?
- What level of secrecy do we require for communication?

Implementing openness

Ideally, a distributed system provides only **mechanisms**:

- Allow (dynamic) setting of caching policies
- Support different levels of trust for mobile code
- Provide adjustable QoS parameters per data stream
- Offer different encryption algorithms

Scale in Distributed Systems [分布式系统中的规模]

Observation

Many developers of modern distributed system easily use the adjective “scalable” without making clear **why** the system actually scales.

Scalability

At least three components:

- Number of users and/or processes (size scalability)
- Maximum distance between nodes (geographical scalability)
- Number of administrative domains (administrative scalability)

Observation

Most systems account only, to a certain extent, for size scalability. The (non)solution: powerful servers. Today, the challenge lies in geographical and administrative scalability.

Scale in Distributed Systems [分布式系统中的规模]

Observation

Many developers of modern distributed system easily use the adjective “scalable” without making clear **why** their system actually scales.

Scalability

At least three components:

- Number of users and/or processes (size scalability)
- Maximum distance between nodes (geographical scalability)
- Number of administrative domains (administrative scalability)

Observation

Most systems account only, to a certain extent, for size scalability. The (non)solution: powerful servers. Today, the challenge lies in geographical and administrative scalability.

Scale in Distributed Systems [分布式系统中的规模]

Observation

Many developers of modern distributed system easily use the adjective “scalable” without making clear **why** their system actually scales.

Scalability

At least three components:

- Number of users and/or processes (size scalability)
- Maximum distance between nodes (geographical scalability)
- Number of administrative domains (administrative scalability)

Observation

Most systems account only, to a certain extent, for size scalability. The (non)solution: powerful servers. Today, the challenge lies in geographical and administrative scalability.

Techniques for Scaling [缩放技术]

Hide communication latencies

Avoid waiting for responses; do something else:

- Make use of **asynchronous communication**
- Have separate handler for incoming response
- **Problem:** not every application fits this model

Techniques for Scaling [缩放技术]

Distribution

Partition data and computations across multiple machines:

- Move computations to clients (Java applets)
- Decentralized naming services (DNS)
- Decentralized information systems (WWW)

Techniques for Scaling [缩放技术]

Replication/caching

Make copies of data available at different machines:

- Replicated file servers and databases
- Mirrored Web sites
- Web caches (in browsers and proxies)
- File caching (at server and client)

Scaling – The Problem [缩放—问题所在]

Observation

Applying scaling techniques is easy, except for one thing:

- Having multiple copies (cached or replicated), leads to **inconsistencies**: modifying one copy makes that copy different from the rest.
- Always keeping copies consistent and in a general way requires **global synchronization** on each modification.
- Global synchronization precludes large-scale solutions.

Observation

If we can tolerate inconsistencies, we may reduce the need for global synchronization, but **tolerating inconsistencies is application dependent**.

Scaling – The Problem [缩放—问题所在]

Observation

Applying scaling techniques is easy, except for one thing:

- Having multiple copies (cached or replicated), leads to **inconsistencies**: modifying one copy makes that copy different from the rest.
- Always keeping copies consistent and in a general way requires **global synchronization** on each modification.
- Global synchronization precludes large-scale solutions.

Observation

If we can tolerate inconsistencies, we may reduce the need for global synchronization, but **tolerating inconsistencies is application dependent**.

Scaling – The Problem [缩放—问题所在]

Observation

Applying scaling techniques is easy, except for one thing:

- Having multiple copies (cached or replicated), leads to **inconsistencies**: modifying one copy makes that copy different from the rest.
- Always keeping copies consistent and in a general way requires **global synchronization** on each modification.
- Global synchronization precludes large-scale solutions.

Observation

If we can tolerate inconsistencies, we may reduce the need for global synchronization, but **tolerating inconsistencies is application dependent**.

Scaling – The Problem [缩放—问题所在]

Observation

Applying scaling techniques is easy, except for one thing:

- Having multiple copies (cached or replicated), leads to **inconsistencies**: modifying one copy makes that copy different from the rest.
- Always keeping copies consistent and in a general way requires **global synchronization** on each modification.
- Global synchronization precludes large-scale solutions.

Observation

If we can tolerate inconsistencies, we may reduce the need for global synchronization, but **tolerating inconsistencies is application dependent**.

Scaling – The Problem [缩放—问题所在]

Observation

Applying scaling techniques is easy, except for one thing:

- Having multiple copies (cached or replicated), leads to **inconsistencies**: modifying one copy makes that copy different from the rest.
- Always keeping copies consistent and in a general way requires **global synchronization** on each modification.
- Global synchronization precludes large-scale solutions.

Observation

If we can tolerate inconsistencies, we may reduce the need for global synchronization, but **tolerating inconsistencies is application dependent**.

Developing Distributed Systems: Pitfalls [开发分布式系统：陷阱]

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing Distributed Systems: Pitfalls [开发分布式系统：陷阱]

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing Distributed Systems: Pitfalls [开发分布式系统：陷阱]

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing Distributed Systems: Pitfalls [开发分布式系统：陷阱]

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing Distributed Systems: Pitfalls [开发分布式系统：陷阱]

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing Distributed Systems: Pitfalls [开发分布式系统：陷阱]

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing Distributed Systems: Pitfalls [开发分布式系统：陷阱]

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing Distributed Systems: Pitfalls [开发分布式系统：陷阱]

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Developing Distributed Systems: Pitfalls [开发分布式系统：陷阱]

Observation

Many distributed systems are needlessly complex caused by mistakes that required patching later on. There are many **false assumptions**:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

摘要 [第二场] Summary [Session 2]

Types of Distributed Systems [分布式系统的类型]

Categorization Basis	Description / Focus	Common Types	Examples
1. Application / Purpose	<i>What is the primary function or domain the system serves?</i>	- Distributed Computing - Distributed Information - Distributed Pervasive (IoT)	- HPC Cluster, Folding@home - Banking Systems, WeChat, Taobao - Smart Home, Sensor Networks
2. Architecture	<i>How are the system's components arranged and connected?</i>	- Centralized - Decentralized - Hybrid	- Traditional Client-Server app - Pure Peer-to-Peer (P2P) file sharing - BitTorrent, Modern Cloud services
3. System / Interaction Model	<i>How do the components communicate and collaborate?</i>	- Client-Server - Peer-to-Peer (P2P) - Multi-tier / N-tier - Publish-Subscribe (Event-based) - Service-Oriented (Microservices)	- Email, Web browsers - Gnutella, early Skype - Web Server → App Server → DB - Apache Kafka, MQTT (IoT) - Amazon, Netflix (E-commerce)
4. Computing Style	<i>How is the actual computation distributed and performed?</i>	- Cluster Computing - Grid Computing - Cloud Computing - Edge Computing - Fog Computing	- Google's original Search Index cluster - Cross-institution scientific resource sharing - AWS (EC2), Alibaba Cloud - Data processed on a local gateway - Intermediate processing between edge and cloud
5. Data / Storage Organization	<i>How is data managed, distributed, and accessed?</i>	- Distributed File Systems - Distributed Databases - Sharded Systems - Replicated Systems	- Google File System (GFS), HDFS - Google Spanner, Apache Cassandra - Database split by geography/user ID - Master-Slave, Multi-Master replication
6. Communication Style	<i>What is the underlying mechanism for message exchange?</i>	- Message Passing - RPC / RMI-based - Stream-based - Event-driven - Shared-Data (Tuple-Space)	- MPI (for HPC), ZeroMQ - Remote service calls (gRPC, Java RMI) - Live video streaming (Twitch) - Event bus architecture - A shared space for data coordination

Distributed Computing Systems

Observation

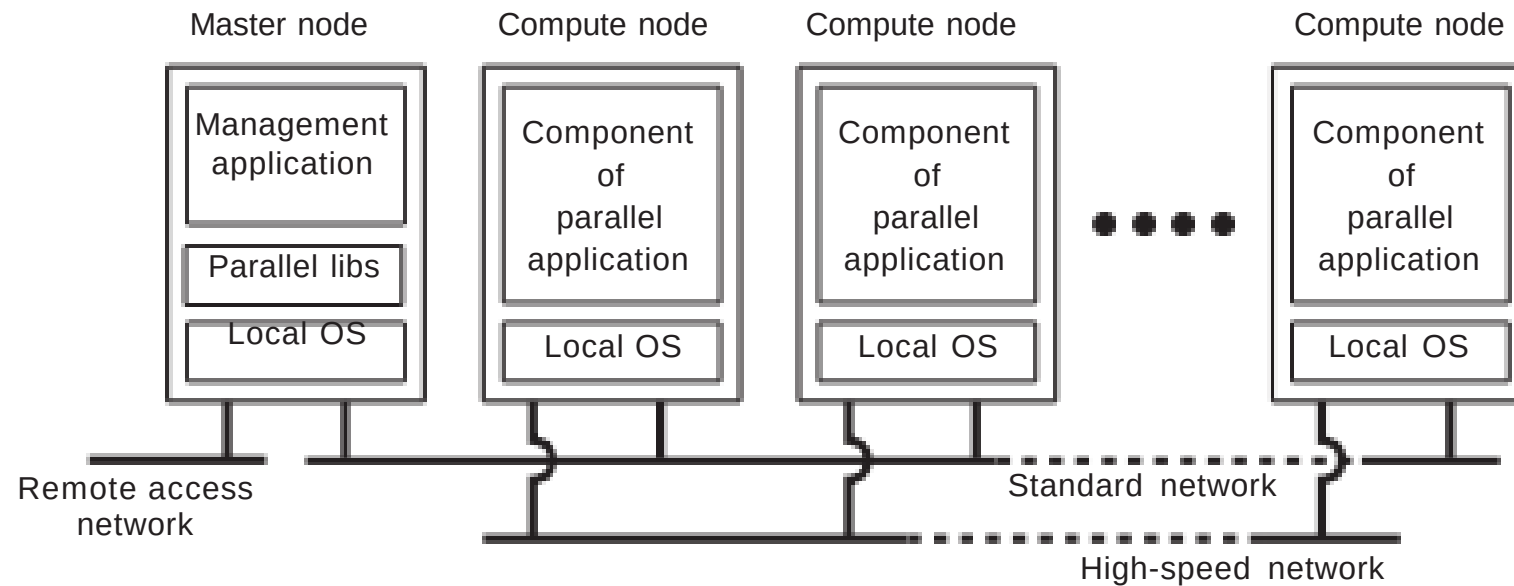
Many distributed systems are configured for **High-Performance Computing**

Cluster Computing

Essentially a group of high-end systems connected through a LAN:

- Homogeneous: same OS, near-identical hardware
- Single managing node

Distributed Computing Systems



Distributed Computing Systems

Grid Computing

The next step: lots of nodes from everywhere:

- Heterogeneous

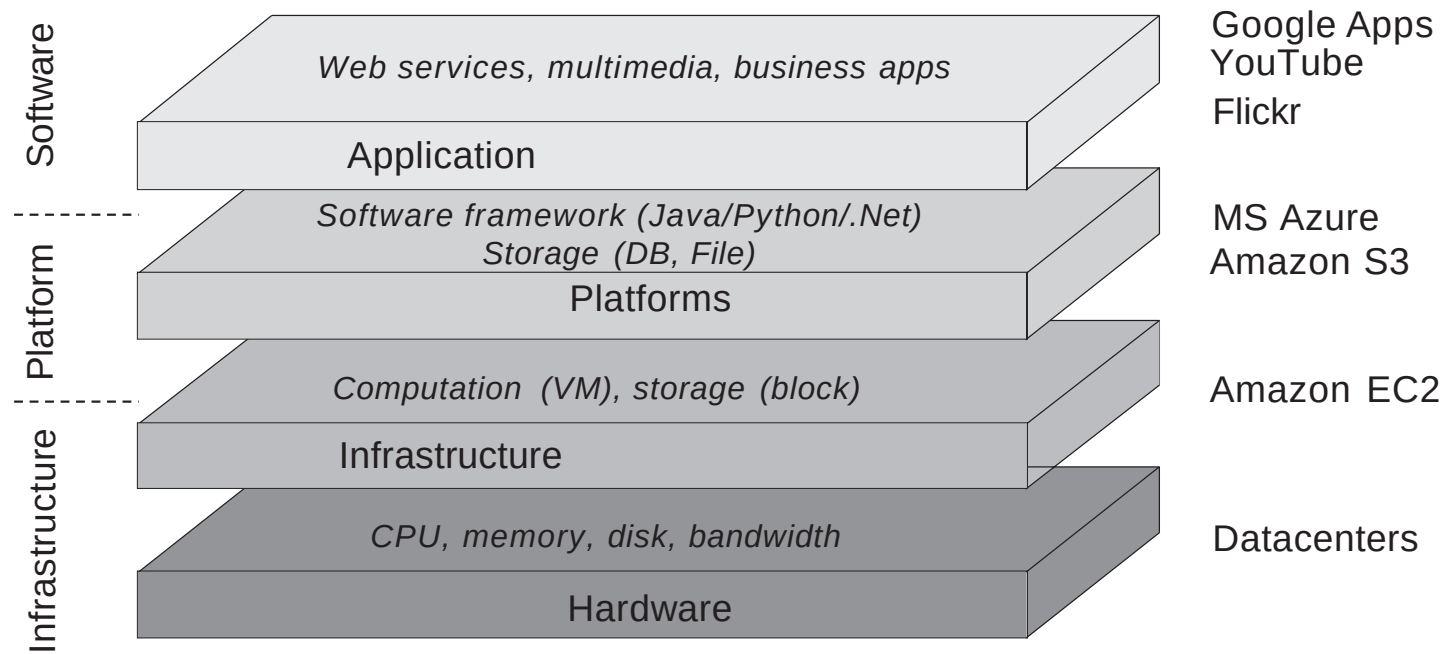
- Dispersed across several organizations

- Can easily span a wide-area network

Note

To allow for collaborations, grids generally use [virtual organizations](#). In essence, this is a grouping of users (or better: their IDs) that will allow for authorization on resource allocation.

Distributed Computing Systems: Clouds



Distributed Computing Systems: Clouds

Cloud computing

Make a distinction between four layer:

- **Hardware**: Processors, routers, power and cooling systems. Customers normally never get to see these.
- **Infrastructure**: Deploys virtualization techniques. Evolves around allocating and managing virtual storage devices and virtual servers.
- **Platform**: Provides higher-level abstractions for storage and such. Example: Amazon S3 storage system offers an API for (locally created) files to be organized and stored in so-**buckets**.
- **Application**: Actual applications, such as office suites (text processors, spreadsheet applications, presentation applications). Comparable to the suite of apps shipped with OSes.

Distributed Information Systems

Observation

The vast amount of distributed systems in use today are forms of traditional information systems, that now **integrate** legacy systems.

Example: Transaction processing systems.

```
BEGIN_TRANSACTION(server, transaction)
READ(transaction, file-1, data)
WRITE(transaction, file-2, data)
newData := MODIFIED(data)
IF WRONG(newData) THEN
    ABORT_TRANSACTION(transaction)
ELSE
    WRITE(transaction, file-2, newData)
    END_TRANSACTION(transaction)
END IF
```

Distributed Information Systems

Observation

The vast amount of distributed systems in use today are forms of traditional information systems, that now **integrate** legacy systems.

Example: Transaction processing systems.

```
BEGIN_TRANSACTION(server, transaction)
READ(transaction, file-1, data)
WRITE(transaction, file-2, data)
newData := MODIFIED(data)
IF WRONG(newData) THEN
    ABORT_TRANSACTION(transaction)
ELSE
    WRITE(transaction, file-2, newData)
    END_TRANSACTION(transaction)
END IF
```

Note

Transactions form an **atomic** operation.

Distributed Information Systems: Transactions

Model

A transaction is a collection of operations on the state of an object (database, object composition, etc.) that satisfies the following properties (**ACID**)

Atomicity: All operations either succeed, or all of them fail. When the transaction fails, the state of the object will remain unaffected by the transaction.

Consistency: A transaction establishes a valid state transition. This does not exclude the possibility of invalid, intermediate states during the transaction's execution.

Isolation: Concurrent transactions do not interfere with each other. It appears to each transaction T that other transactions occur either **before** T , or **after** T , but never both.

Durability: After the execution of a transaction, its effects are made permanent: changes to the state survive failures.

Distributed Information Systems: Transactions

Model

A transaction is a collection of operations on the state of an object (database, object composition, etc.) that satisfies the following properties (ACID)

Atomicity: All operations either succeed, or all of them fail. When the transaction fails, the state of the object will remain unaffected by the transaction.

Consistency: A transaction establishes a valid state transition. This does not exclude the possibility of invalid, intermediate states during the transaction's execution.

Isolation: Concurrent transactions do not interfere with each other. It appears to each transaction T that other transactions occur either **before** T , or **after** T , but never both.

Durability: After the execution of a transaction, its effects are made permanent: changes to the state survive failures.

Distributed Information Systems: Transactions

Model

A transaction is a collection of operations on the state of an object (database, object composition, etc.) that satisfies the following properties (ACID)

Atomicity: All operations either succeed, or all of them fail. When the transaction fails, the state of the object will remain unaffected by the transaction.

Consistency: A transaction establishes a valid state transition. This does not exclude the possibility of invalid, intermediate states during the transaction's execution.

Isolation: Concurrent transactions do not interfere with each other. It appears to each transaction T that other transactions occur either **before** T , or **after** T , but never both.

Durability: After the execution of a transaction, its effects are made permanent: changes to the state survive failures.

Distributed Information Systems: Transactions

Model

A transaction is a collection of operations on the state of an object (database, object composition, etc.) that satisfies the following properties (ACID)

Atomicity: All operations either succeed, or all of them fail. When the transaction fails, the state of the object will remain unaffected by the transaction.

Consistency: A transaction establishes a valid state transition. This does not exclude the possibility of invalid, intermediate states during the transaction's execution.

Isolation: Concurrent transactions do not interfere with each other. It appears to each transaction T that other transactions occur either **before** T , or **after** T , but never both.

Durability: After the execution of a transaction, its effects are made permanent: changes to the state survive failures.

Distributed Information Systems: Transactions

Model

A transaction is a collection of operations on the state of an object (database, object composition, etc.) that satisfies the following properties (ACID)

Atomicity: All operations either succeed, or all of them fail. When the transaction fails, the state of the object will remain unaffected by the transaction.

Consistency: A transaction establishes a valid state transition. This does not exclude the possibility of invalid, intermediate states during the transaction's execution.

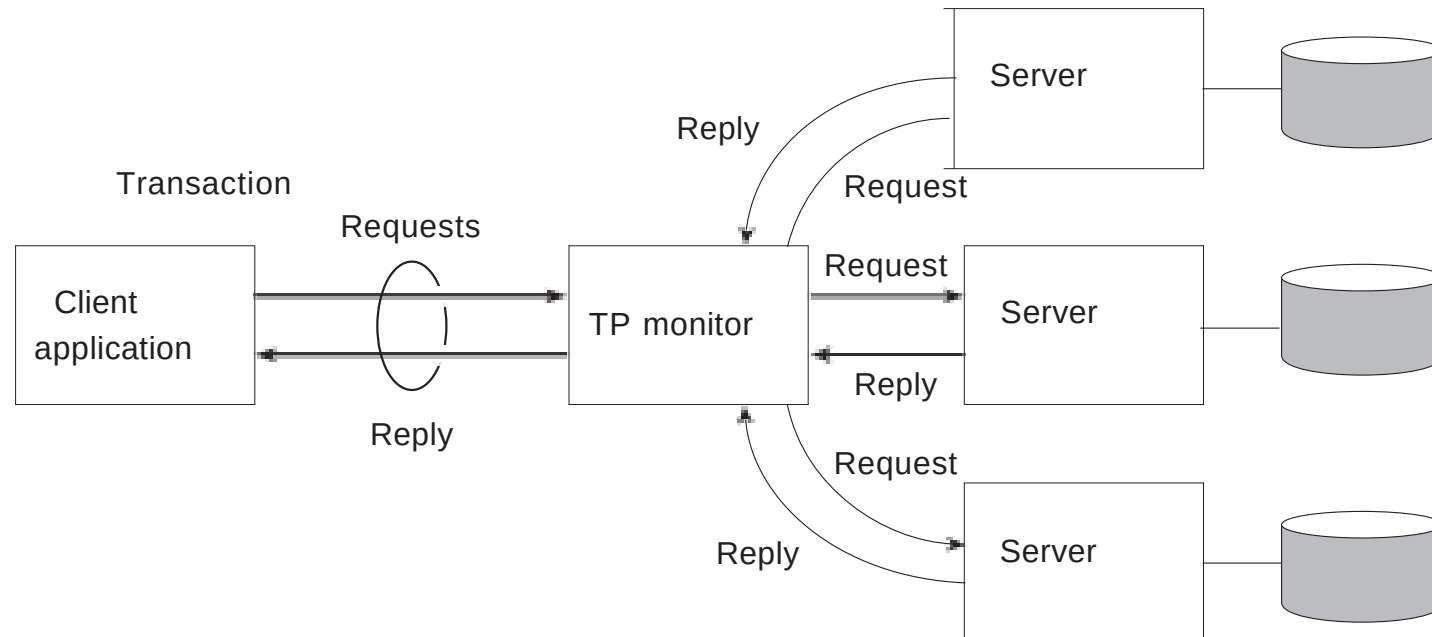
Isolation: Concurrent transactions do not interfere with each other. It appears to each transaction T that other transactions occur either **before** T , or **after** T , but never both.

Durability: After the execution of a transaction, its effects are made permanent: changes to the state survive failures.

Transaction Processing Monitor

Observation

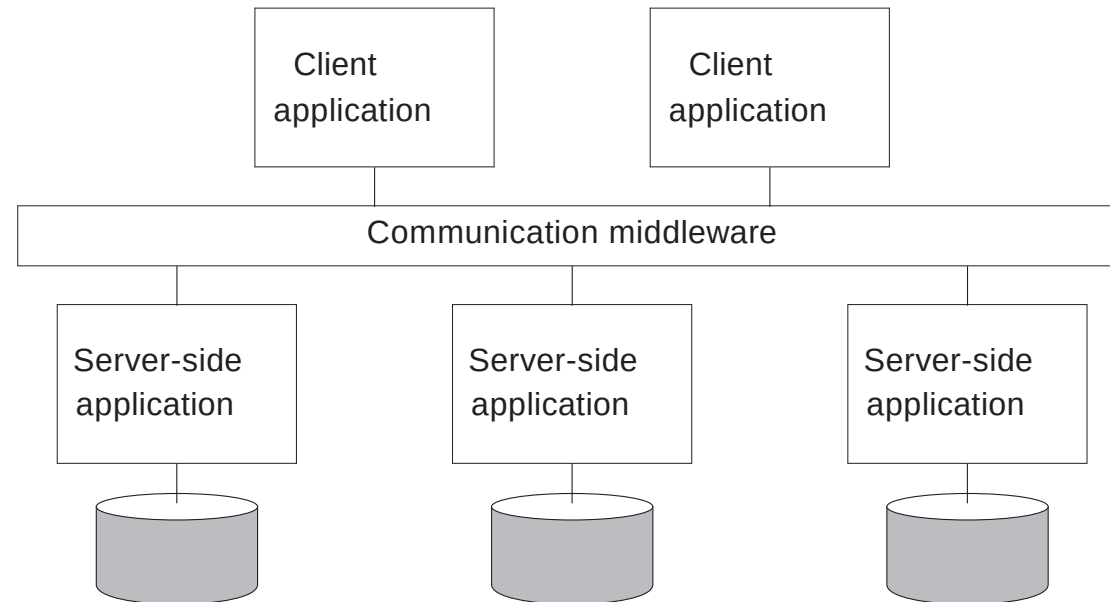
In many cases, the data involved in a transaction is distributed across several servers. A **TP Monitor** is responsible for coordinating the execution of a transaction



Distr. Info. Systems: Enterprise Application Integration

Problem

A TP monitor doesn't separate apps from their databases. Also needed facilities for direct communication between apps.



Distributed Pervasive Systems

Observation

Emerging next-generation of distributed systems in which nodes are small, mobile, and often embedded in a larger system.

Some requirements

- **Contextual change:** The system is part of an environment in which changes should be immediately accounted for.
- **Ad hoc composition:** Each node may be used in a very different ways by different users. Requires ease-of-configuration.
- **Sharing is the default:** Nodes come and go, providing sharable services and information. Calls again for simplicity.

Note

Pervasiveness and distribution transparency: a good match?

Distributed Pervasive Systems

Observation

Emerging next-generation of distributed systems in which nodes are small, mobile, and often embedded in a larger system.

Some requirements

- **Contextual change:** The system is part of an environment in which changes should be immediately accounted for.
- **Ad hoc composition:** Each node may be used in a very different ways by different users. Requires ease-of-configuration.
- **Sharing is the default:** Nodes come and go, providing sharable services and information. Calls again for simplicity.

Note

Pervasiveness and distribution transparency: a good match?

Pervasive Systems: Examples

Home systems

Should be completely self-organizing:

- There should be no system administrator
- Simplest solution: a centralized **home box**?

Monitoring a person

Devices are physically close to a person:

Where and how should monitored data be stored?

How can we prevent loss of crucial data?

What is needed to generate and propagate alerts?

How can security be enforced?

How can environment provide online feedback?

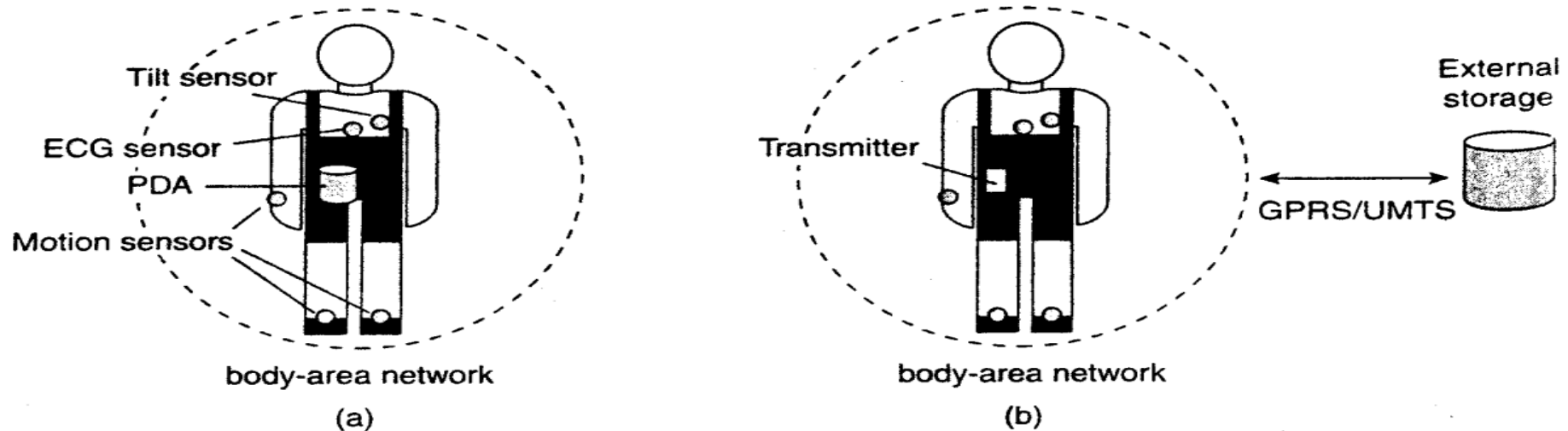
Pervasive Systems: Examples

Home systems

Should be completely self-organizing:

- There should be no system administrator
- Simplest solution: a centralized **home box**?

Monitoring a person



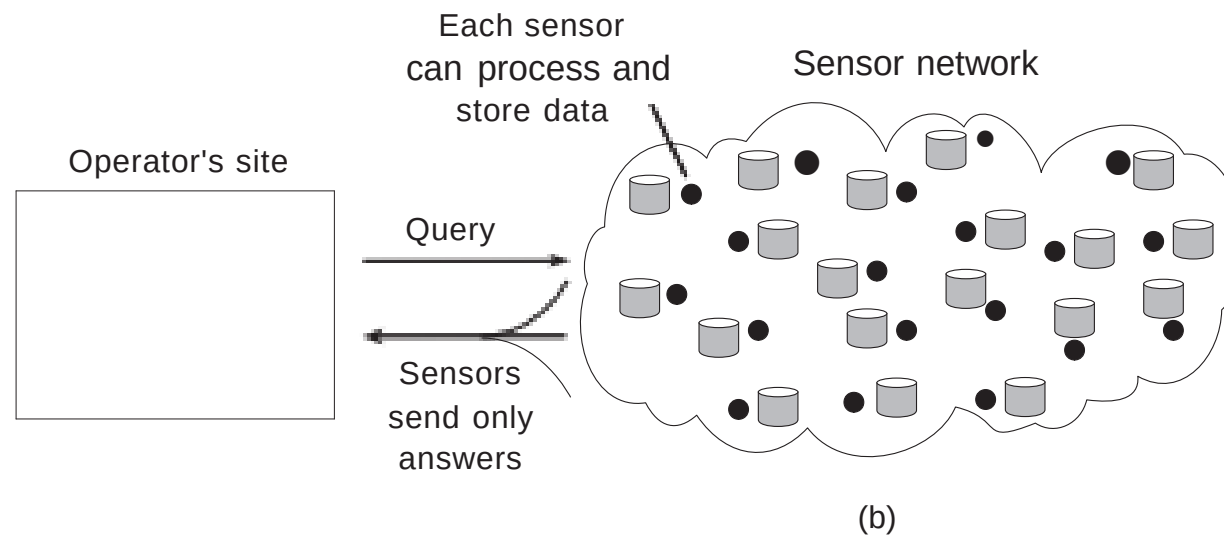
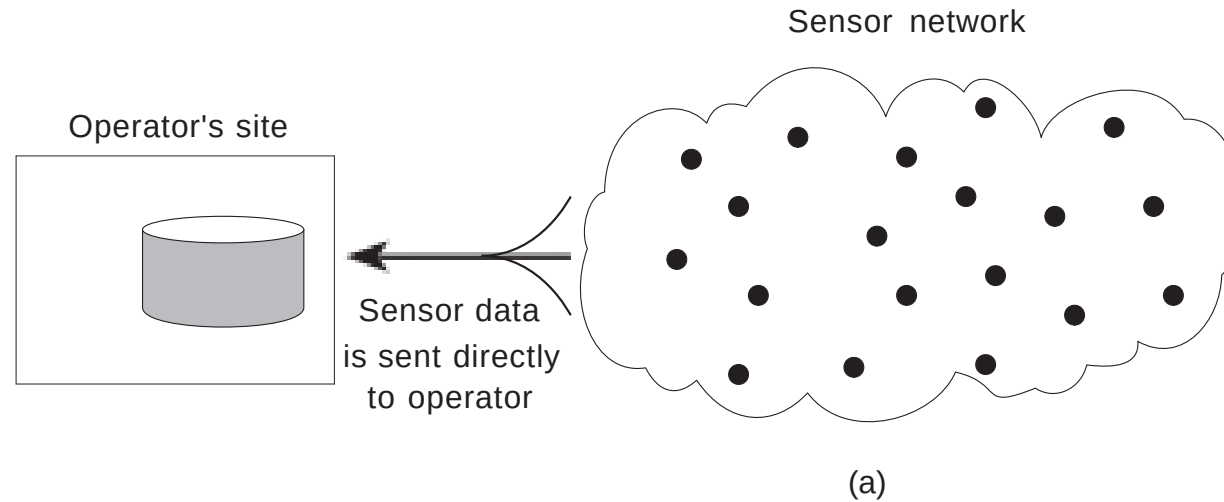
Sensor networks

Characteristics

The **nodes** to which sensors are attached are:

- Many (10s-1000s)
- Simple (small memory/compute/communication capacity)
- Often battery-powered (or even battery-less)

Sensor networks as distributed systems



Summary [Session 3] 摘要 [第三场]

It is crucial to understand that these are different perspectives, not mutually exclusive definitions. The same system is often described by many of these classifications simultaneously:

Example: Taobao (E-commerce Platform)

- ☐ By Application: Distributed Information System
- ☐ By Architecture: Hybrid / Multi-tier
- ☐ By Model: Microservices-based
- ☐ By Computing Style: Cloud Computing
- ☐ By Data Organization: Heavily Sharded and Replicated
- ☐ By Communication: Mix of RPC and Event-driven

Summary [Session 3] 摘要 [第三场]

DECODING A COMPLEX SYSTEM: TAOBAO THROUGH MULTIPLE LENSES

1. BY APPLICATION: DISTRIBUTED INFORMATION SYSTEM



- Manages massive data: listings, profiles, orders
- Processes transactions

2. BY ARCHITECTURE: HYBRID / MULTI-TIER

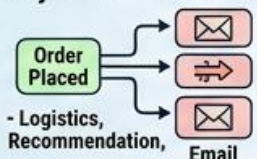


6. BY COMMUNICATION: MIX OF RPC & EVENT-DRIVEN

RPC Synchronous Call



EVENT-DRIVEN Asynchronous Events



3. BY SYSTEM MODEL: MICROSERVICES-BASED



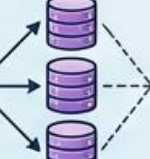
- Thousands of independent services
- Use APIs to coordinate

5. BY DATA ORGANIZATION: HEAVILY SHARDED & REPLICATED

Data Partitioned



Data Copied



- Notes:
- High Availability
- Improved Performance

4. BY COMPUTING STYLE: CLOUD COMPUTING



- Runs on massive, elastic cloud infrastructure (Alibaba Cloud)
- Scales resources dynamically (e.g., Singles' Day)

Project Discussion [项目讨论]

Documents Extracted from Scopus [摘自Scopus的文件]

TITLE-ABS-KEY ("Distributed Systems")
52,233 documents found



AND
(LIMIT-TO (PUBYEAR , "2022") OR LIMIT-TO (PUBYEAR , "2023") OR LIMIT-TO (PUBYEAR , "2024") OR LIMIT-TO (PUBYEAR , "2025") OR LIMIT-TO (PUBYEAR , "2026"))
7,839 documents found



AND
(LIMIT-TO (LANGUAGE , "English"))
7,683 documents found



AND
(LIMIT-TO (DOCTYPE , "cp") OR LIMIT-TO (DOCTYPE , "ar") OR LIMIT-TO (DOCTYPE , "ch") OR LIMIT-TO (DOCTYPE , "re") OR LIMIT-TO (DOCTYPE , "bk") OR LIMIT-TO (DOCTYPE , "sh"))
7,473 documents found

Documents Extracted from Scopus [摘自Scopus的文件]

The screenshot shows the Scopus search interface. At the top, the Scopus logo is on the left, and navigation links for Search, Sources, SciVal, and user icons are on the right. The search bar contains the query "Distributed Systems". Below the search bar, there are tabs for Documents, Preprints, and Secondary documents. The search results show 7,473 documents found. On the left, there are filters for Year (Limited to 2025, 2024, 2022, 2023, 2026) and Subject area. The main results table lists two documents:

Document title	Authors	Source	Year	Citations
1 Distributed carbon emission prediction in edge environment considering spatial-temporal context factors	Li Y., Mokarram M.J.	Journal of Cloud Computing, 15(1), 41	2026	0
2 Continuous software monitoring backed by process mining: a systematic literature review	Stepanov E., Miltshuk A.	International Journal of Data Science and Analytics, 22(1), 6	2026	0

Documents Extracted from Scopus [摘自Scopus的文件]

The screenshot displays the Scopus search interface. At the top, the Scopus logo is on the left, and navigation links for Search, Sources, SciVal, and user icons are on the right. The search area features two input boxes for 'Search within' (Article title, Abstract, Keywords) and 'Search documents' (e.g., "Distributed Systems", "Fault Tolerance"). Below these are options for 'AND' and 'Add search field'. A 'Search' button is present. The results section shows '549 documents found' with options to 'Refine search', 'Export', 'Download', 'Citation overview', and 'Show all abstracts'. The results are sorted by 'Date (newest)'. The first two results are listed below.

	Document title	Authors	Source	Year	Citations
☐ 1	A trustworthy vehicle-to-vehicle power trading scheme based on spatio temporal network and blockchain	Qi J., Zhao Z., Luo Z., ... Wang H., Liu Y.	Energy Reports, 15, 109094	2026	0
☐ 2	Prioritized-MVBA: A New Approach to Design an Optimal Asynchronous Byzantine Agreement Protocol	Sony N.S., Ding X.	International Journal of Information Security	2026	0

Documents Extracted from Scopus [摘自Scopus的文件]

The screenshot shows the Scopus search interface. At the top, there's a navigation bar with the Scopus logo, a search bar, and links for Sources, SciVal, and user settings. Below the navigation bar, there's a search area with three search fields, each containing a search query: "Distributed Systems", "Fault Tolerance", and "Scalability". The search results section shows 156 documents found. A table of results is displayed, with columns for Document title, Authors, Source, Year, and Citations. The first result is "A formal framework for fault tolerance in hybrid scientific workflows" by Mulone A, Medic D, Colonnelli I, and Aldinucci M, published in Future Generation Computer Systems in 2026, with 176,108 citations. The interface also includes options to refine the search, export results, and view the abstract.

Scopus

Search Sources SciVal ?

Advanced query

Search within Article title, Abstract, Keywords

Search documents "Distributed Systems"

AND

Search within Article title, Abstract, Keywords

Search documents "Fault Tolerance"

AND

Search within Article title, Abstract, Keywords

Search documents "Scalability"

Save search Set search alert

+ Add search field

Reset Search

Documents Preprints Secondary documents

156 documents found

Analyze results

Refine search

Search within results

Filters Clear all

Year Clear (5)

☐ All ☐ Export ☐ Download ☐ Citation overview ☐ More

Show all abstracts Sort by Date (newest)

	Document title	Authors	Source	Year	Citations
☐ 1	A formal framework for fault tolerance in hybrid scientific workflows	Mulone A, Medic D, Colonnelli I, Aldinucci M	Future Generation Computer Systems	2026	0

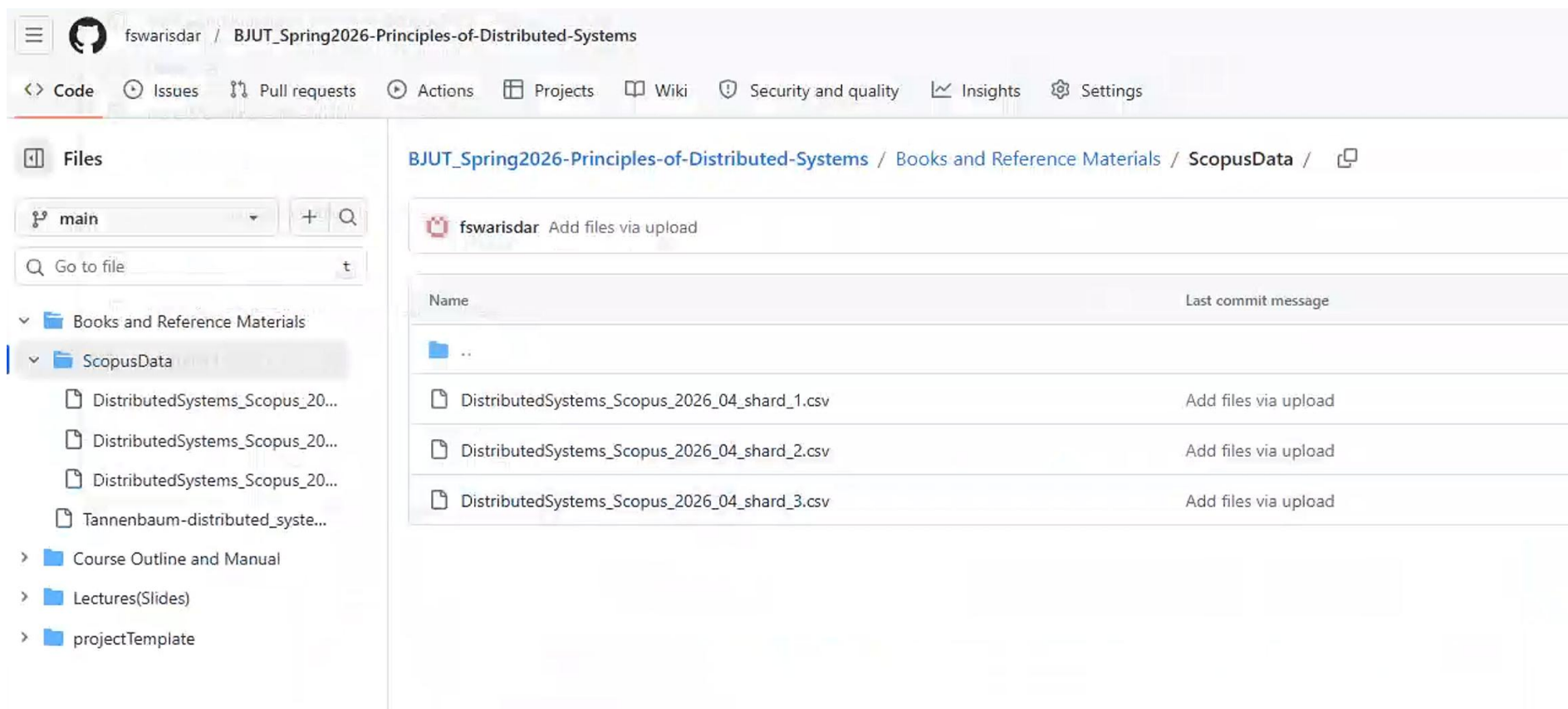
ar • Open access

Show abstract ☐ Find in View at Publisher Related documents

北京工业大学 分布式系统的原理
Beijing University of Technology, Principles of Distributed Systems

Documents Extracted from Scopus [摘自Scopus的文件]

- ❑ The extracted data can be downloaded from: [github](#)
- ❑ Books and Reference Materials/ScopusData



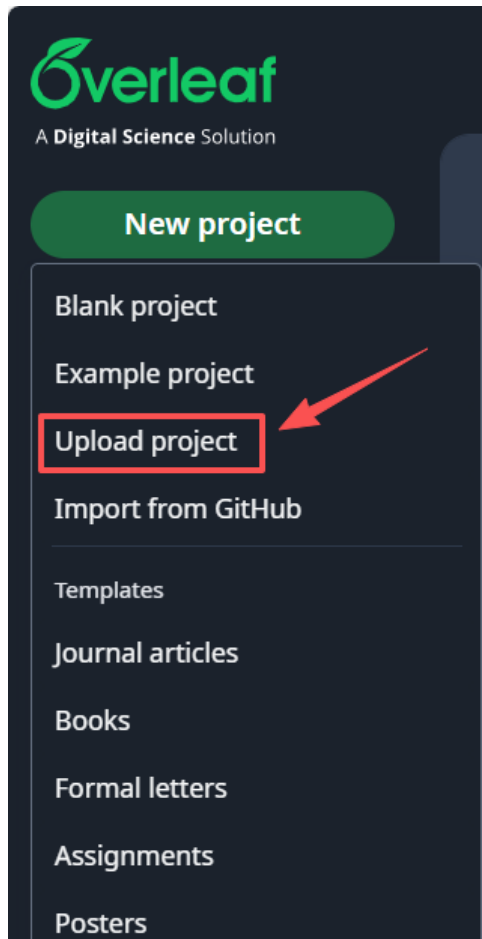
Creating an Overleaf Project [创建 Overleaf 项目]

1. Select your group members(a group can have 2 to 3 members).
2. Identify the areas of interest and select the project/paper title.
3. Every group has to create one overleaf project.
 - ☐ The template can be downloaded from: [github](#)
 - ☐ Turn on link sharing and copy/share the link.
 - ☐ Register your group using following QR.
 - ☐ Set the information required (Project/Paper Title, Authors Names and email addresses, abstract etc.)



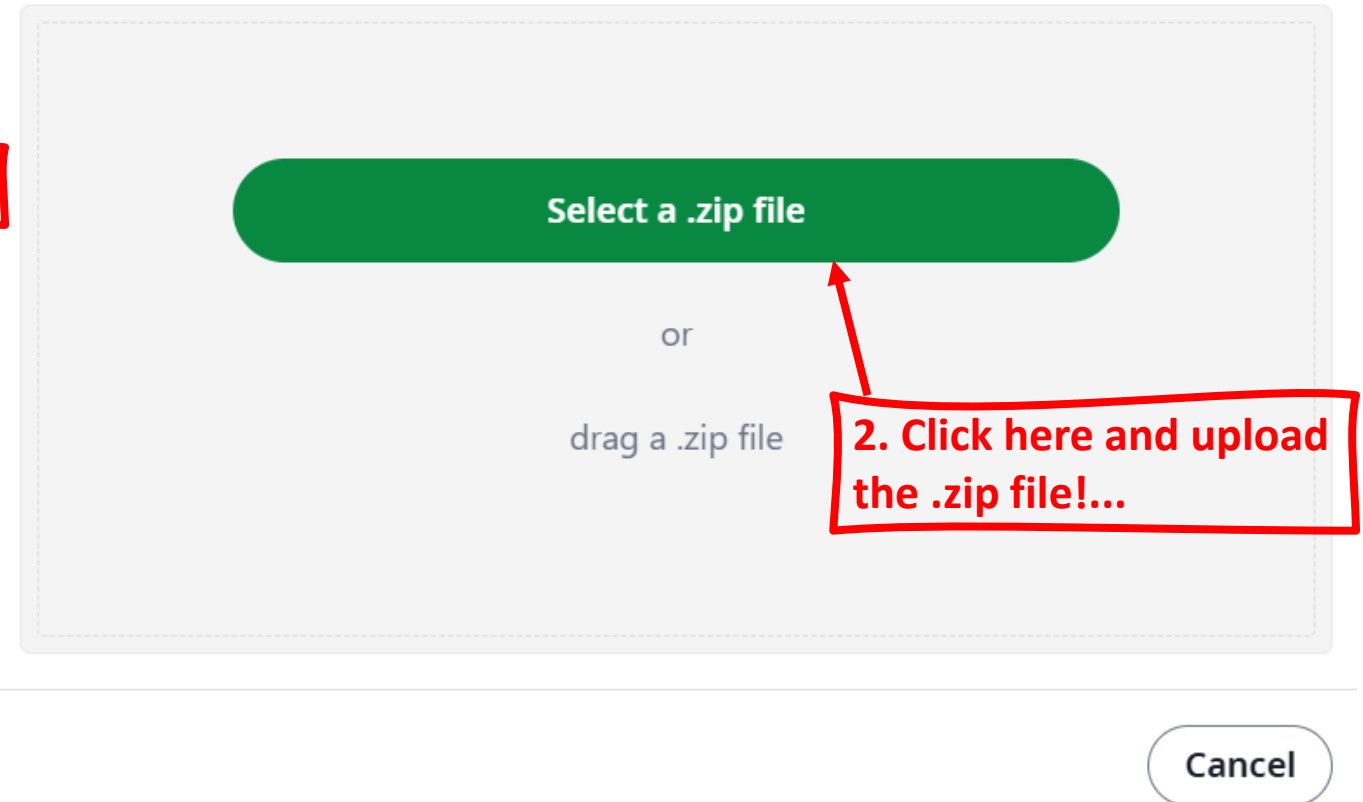
Creating an Overleaf Project [创建 Overleaf 项目]

<https://www.overleaf.com/>



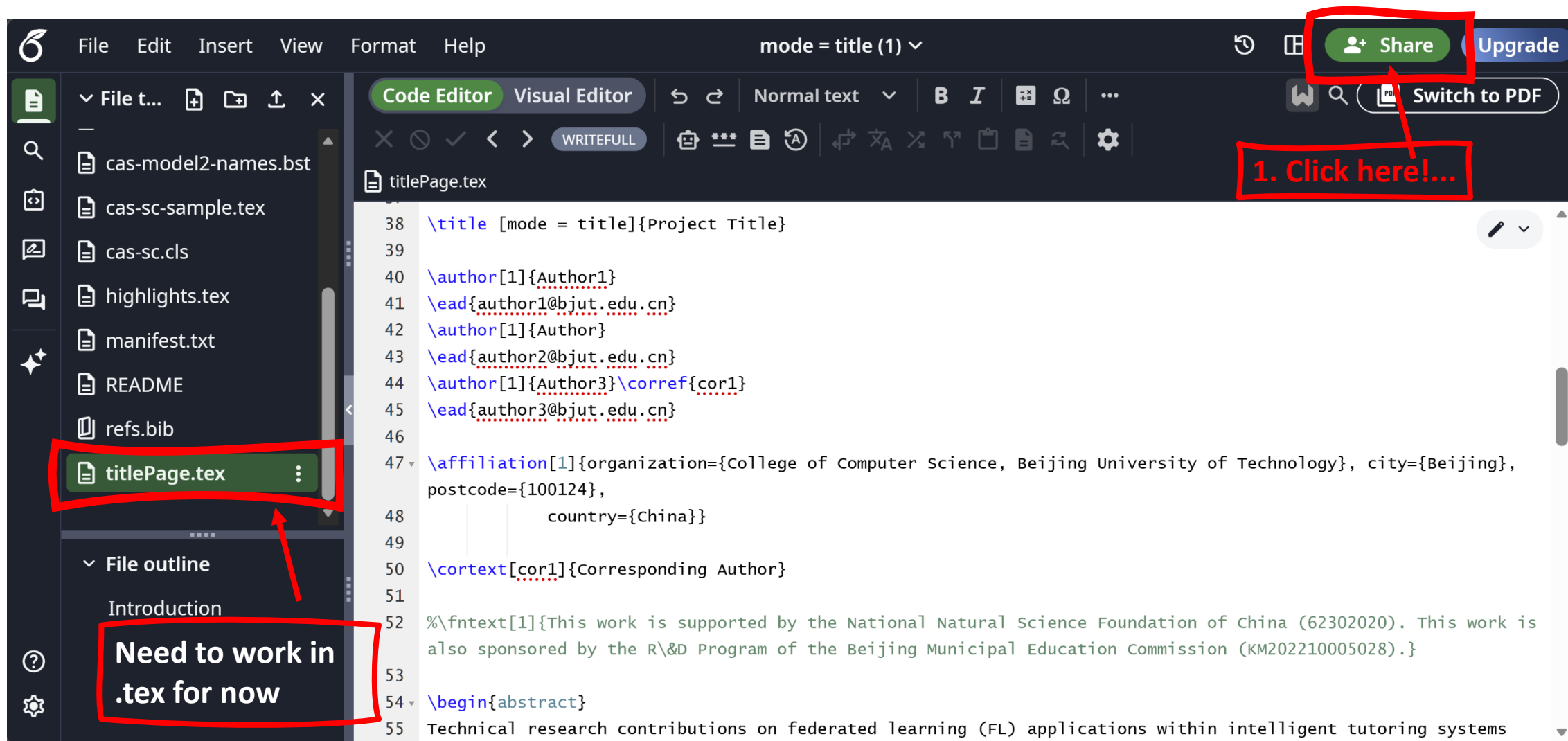
1. Click here!...

Upload Zipped Project



2. Click here and upload the .zip file!...

Creating an Overleaf Project [创建 Overleaf 项目]



Creating an Overleaf Project [创建 Overleaf 项目]

Share Project

×

Add email address

Separate multiple email addresses using the comma (,) character.

Editor



Invite

Link sharing is off [Turn on link sharing](#) ?



fahad.sabah@bjut.edu.cn

Owner

Close

Creating an Overleaf Project [创建 Overleaf 项目]

Share Project

×

Add email address

Separate multiple email addresses using the comma (,) character.

Editor



Invite

Link sharing is on [Turn off link sharing](#) ? ^

Anyone with this link can edit this project

<https://www.overleaf.com/9387982322ksggzpxnzmbw#42d19a>

Copy

Anyone with this link can view this project

<https://www.overleaf.com/read/pkbgwncgmzxy#bf59a3>

Copy

fahad.sabah@bjut.edu.cn

Owner

**Copy and share
this link with
your group
members and for
registration of
your project**

Creating an Overleaf Project [创建 Overleaf 项目]

The screenshot shows the Overleaf web editor interface. The left sidebar contains a file explorer with several files, including `titlePage.tex`, which is highlighted with a red box and an arrow pointing to it. Below the file explorer, a red box contains the text "Need to work in .tex for now". The main editor area displays the LaTeX code for `titlePage.tex`. Red boxes and arrows highlight specific parts of the code:

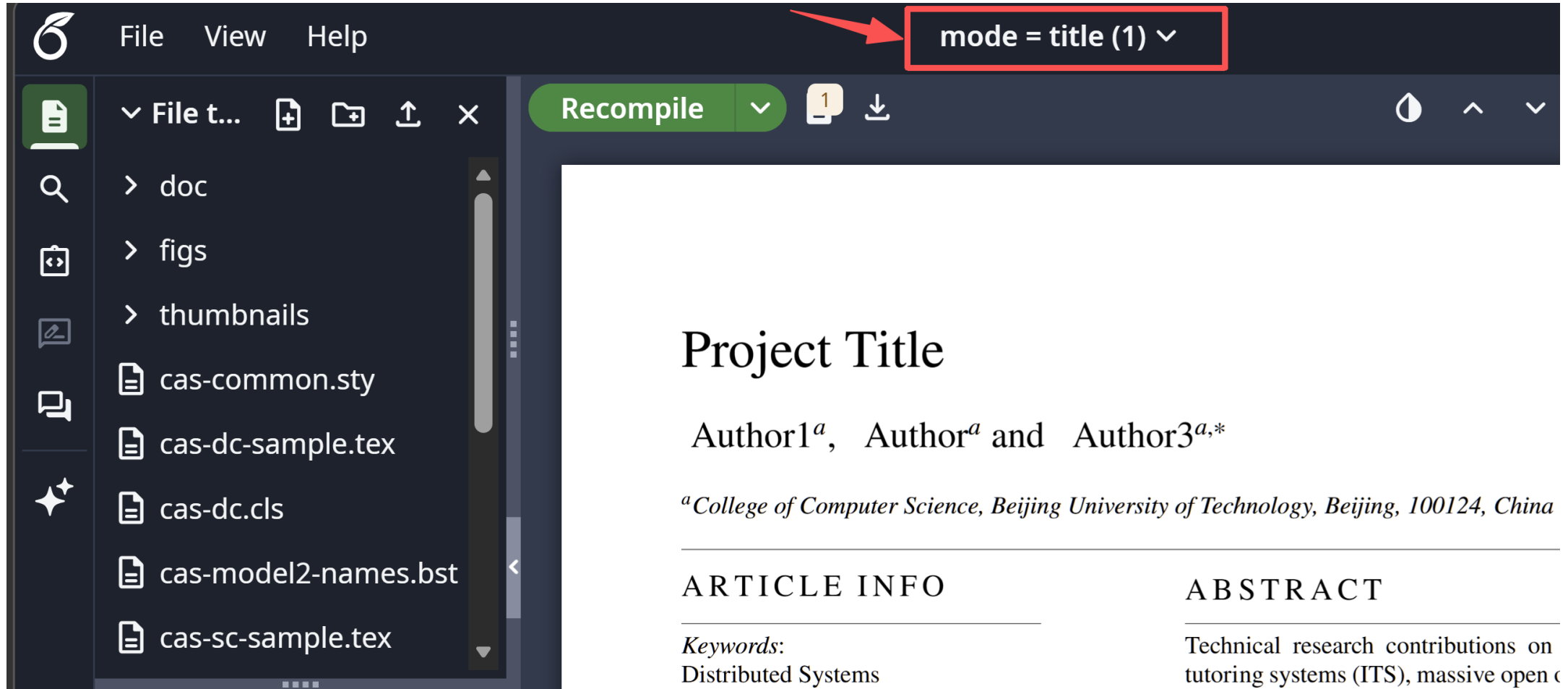
- A red box around the `\title` command is labeled "2. Set the project title!..."
- A red box around the `\author` and `\lead` commands is labeled "3. Set the authors details!..."
- A red box around the `\begin{abstract}` command is labeled "4. Write an abstract(will finalize it later)!..."

 In the top right corner, a red box around the "Share" button is labeled "1. Click here!...". The interface also shows a "Switch to PDF" button and a "mode = title (1)" dropdown.

```
38 \title [mode = title]{Project Title}
39
40 \author[1]{Author1}
41 \lead{author1@bjut.edu.cn}
42 \author[1]{Author}
43 \lead{author2@bjut.edu.cn}
44 \author[1]{Author3}\corref{cor1}
45 \lead{author3@bjut.edu.cn}
46
47 \affiliation[1]{organization={College of Computer Science, Beijing University of Technology}, city={Beijing},
48               postcode={100124},
49               country={China}}
50 \cortext[cor1]{Corresponding Author}
51
52 %\fntext[1]{This work is supported by the National Natural Science Foundation of China (62302020). This work is
53            also sponsored by the R\&D Program of the Beijing Municipal Education Commission (KM202210005028).}
54 \begin{abstract}
55 Technical research contributions on federated learning (FL) applications within intelligent tutoring systems
```

Creating an Overleaf Project [创建 Overleaf 项目]

1. Overleaf Project Title (Group ID).



Some Conferences and Journals to Follow ...

- [PODC](#)

PODC 2026 will be held at Royal Holloway, University of London in Egham, England, on July 6-10, 2026.

- [DISC](#)

The International Symposium on Distributed Computing (DISC) is an international forum on the theory, design, analysis, implementation and application of distributed systems and networks.

DISC 2026 will be held in Rome, Italy on November 9-13, 2026.

- [ICDCS](#)

IEEE International Conference on Distributed Computing Systems

The 46th IEEE ICDCS 2026 will be held in person from June 22–25, 2026.

- [OSDI/SOSP](#)

USENIX Symposium on Operating Systems Design and Implementation (OSDI '26) will take place on July 13–15, 2026, at The Westin Seattle in Seattle, WA, USA.

- [ASPLOS](#)

International Conference on Architectural Support for Programming Languages and Operating Systems **already held in March 2026, Pittsburgh, USA**

- [IEEE Transactions on Parallel and Distributed Systems](#)

- [ACM Transactions on Computer Systems](#)

Final takeaway for Week 1 第一周总结

Distributed systems are about teamwork among computers — and trade-offs among:

Performance

Scalability

Availability

Replication

Agreement

Caching

Which is more important for a given app: very strong correctness, very low delay, or extreme scale? The answer depends on the application.